

System monitorowania warunków środowiskowych EYE

Grzegorz Lademann, Kamil Grzybek, Jakub Lidzbarski, Jarosław Wojciuk

Gdynia 10.06.2025

Opis projektu



Projekt EYE to system monitorowania warunków środowiskowych w pomieszczeniach przeznaczonych do nauki, pracy lub wypoczynku.



Jego celem jest poprawa komfortu poprzez reagowanie na przekroczenie ustalonych wartości parametrów.



System odczytuje dane z czujników, przesyła je do centralnego serwera z wielu urządzeń, umożliwia ich wizualizację oraz definiowanie zdarzeń wyzwalanych przez określone warunki środowiskowe.

Cel Projektu

Celem projektu EYE jest odczytywanie danych z czujników w czasie rzeczywistym, ich akwizycja oraz wizualizacja. System umożliwia organizację przestrzeni na pomieszczenia, z przypisanymi do nich urządzeniami pomiarowymi. Dostęp do danych z poszczególnych pomieszczeń mają wybrani użytkownicy. Odczyty prezentowane są zarówno w formie liczbowej, jak i graficznej (wykresy). Dane zbierane są przez mikrokontroler, który odczytuje wartości z czujników i przesyła je w formacie JSON przez REST API do serwera Django, odpowiedzialnego za ich zapisywanie i prezentację.



Założenia

- Komunikacja ESP32-S3 poprzez WiFi z serwerem za pomocą REST API i szyfrowaniu danych
- Oprogramowanie ESP32 w C++
- W pełni konfigurowalny System ESP32 poprzez interfejs szeregowy
- Aplikacja webowa z mechanizmami bezpieczeństwa
- Obsługa wielu urządzeń – serwer przyjmuje dane od wielu urządzeń pomiarowych jednocześnie
- Silnik baz danych MySQL, Backend i Frontend Django



Wybór technologii i platformy



Django – Framework Strony Internetowej



MySQL – Silnik Baz Danych



FireBeetle 2 ESP32-S3-WROOM



OrangePi Zero 3 4GB – linux server



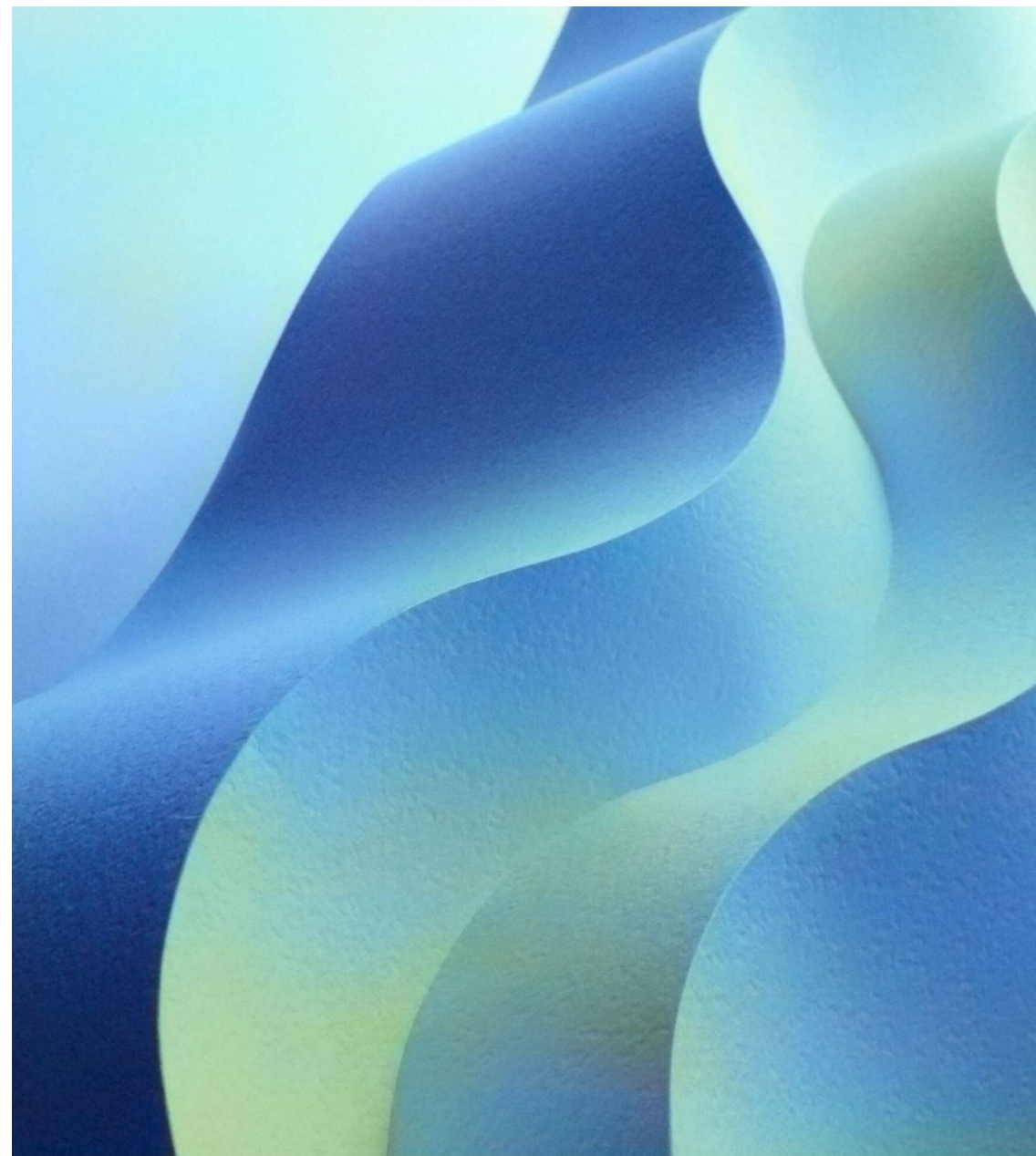
ENS160 – czujnik jakości powietrza



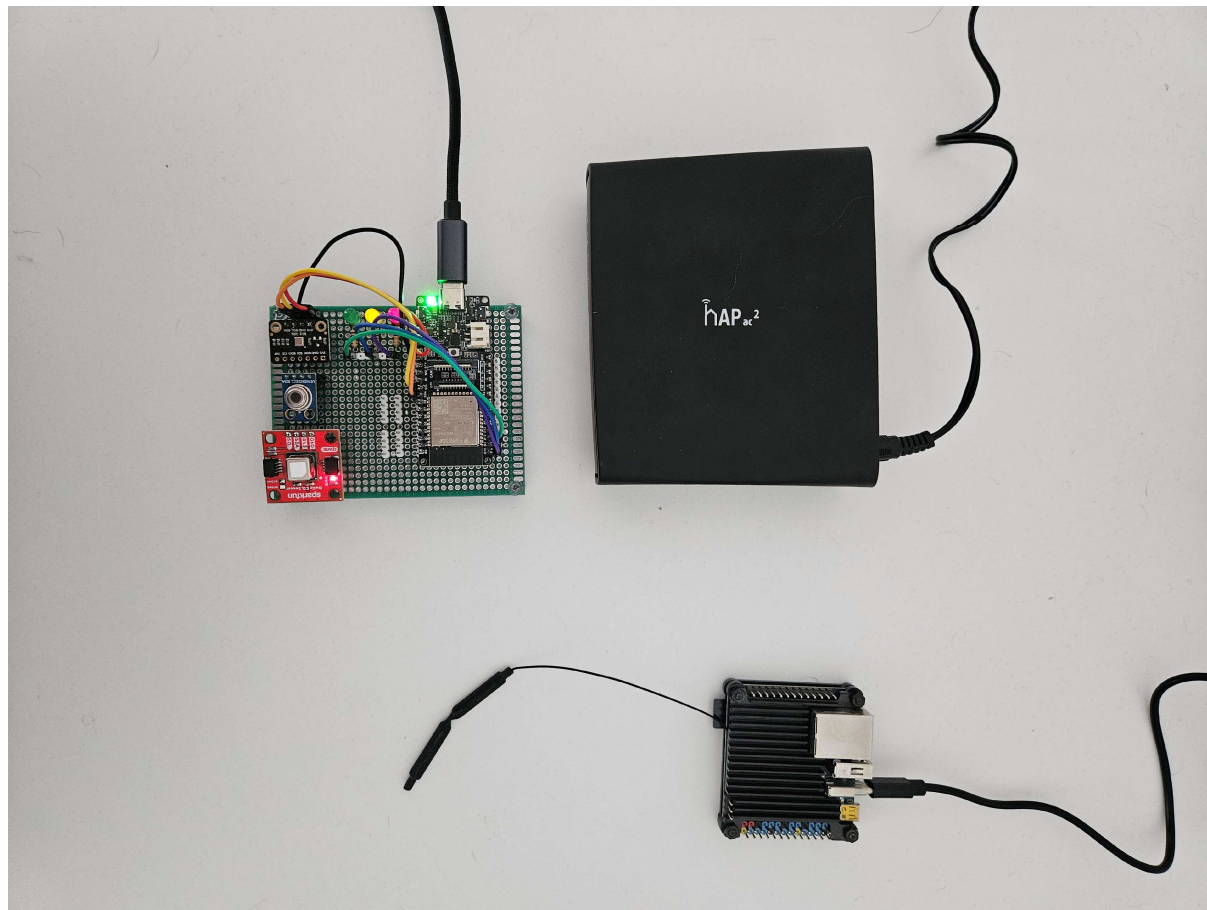
GY-906 – bezkontaktowy czujnik temperatury



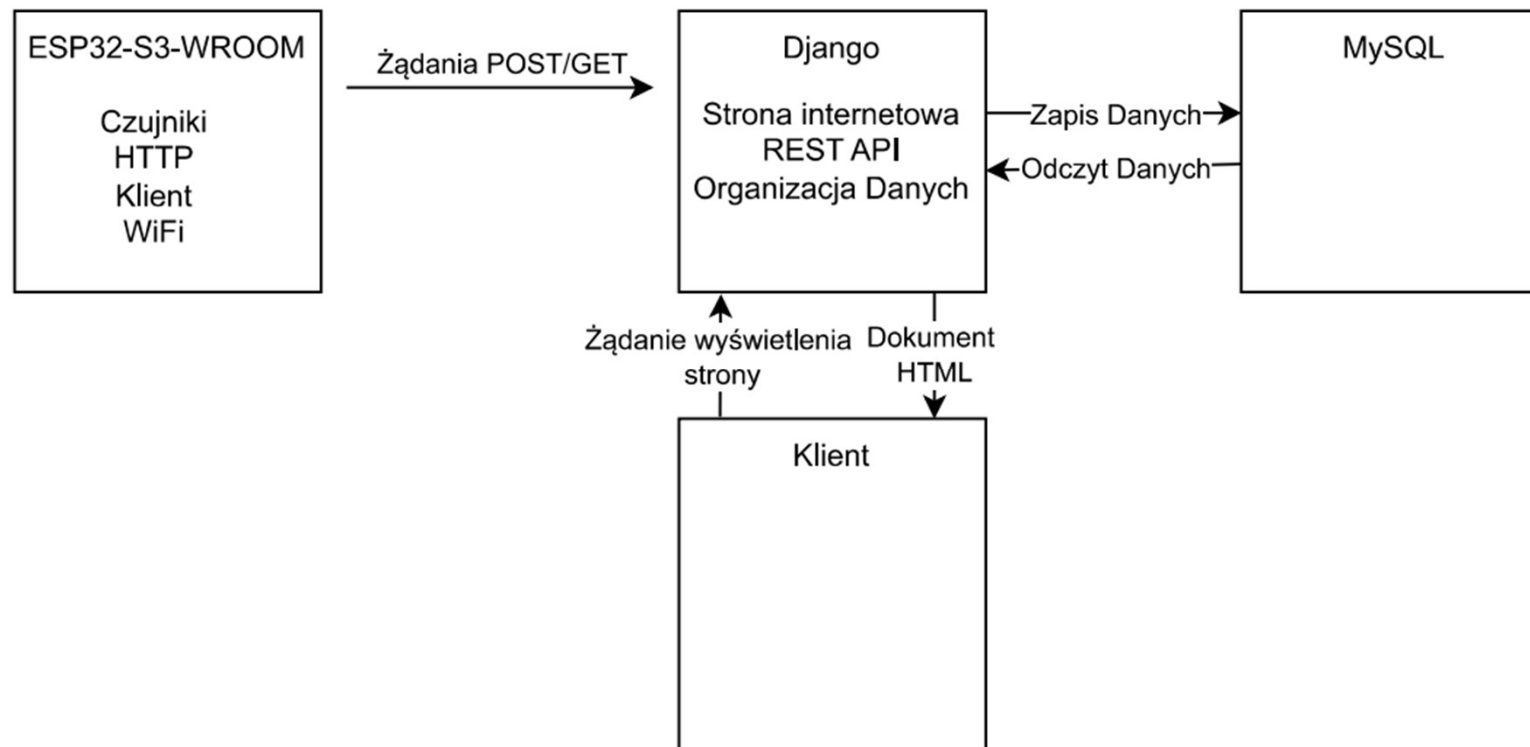
Sen-22396 – czujnik CO2, wilgotności i temperatury



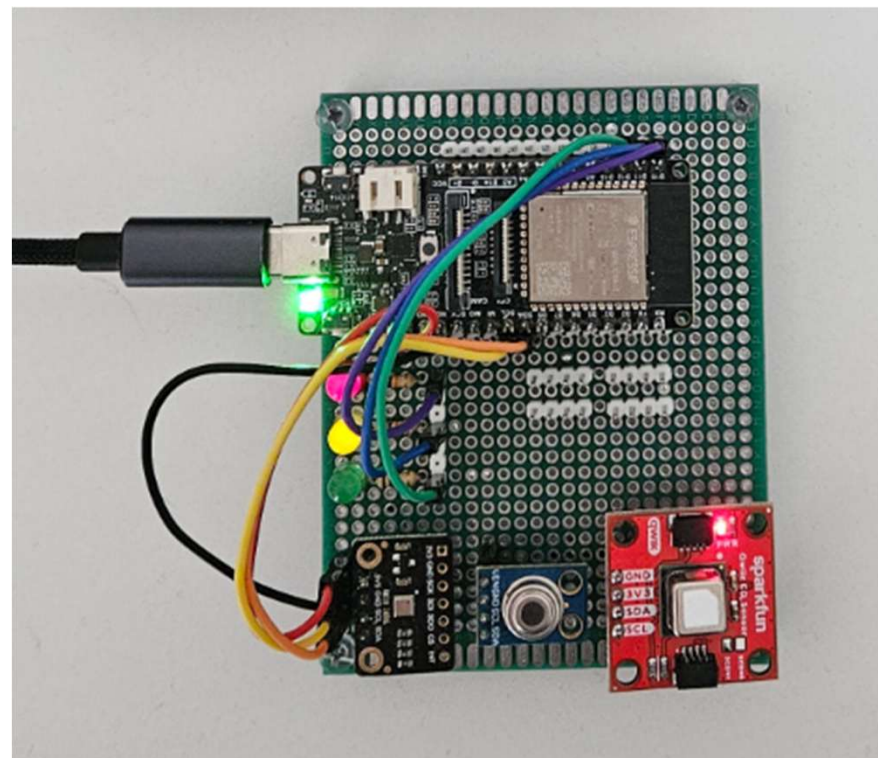
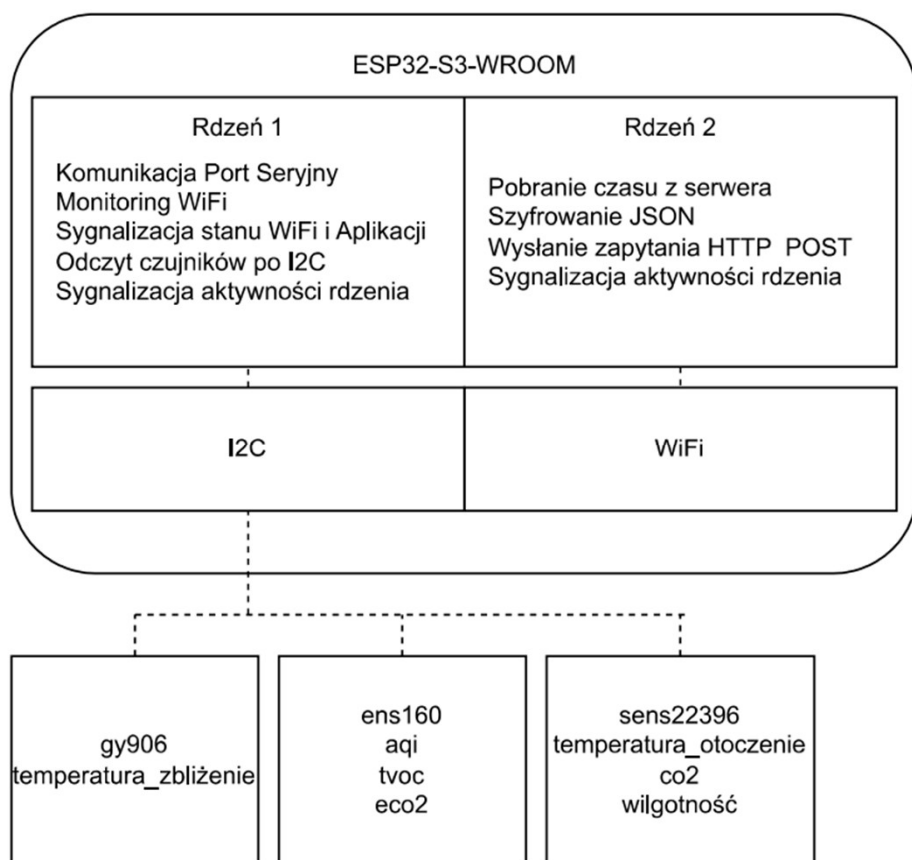
Architektura Systemu - Ogólnie



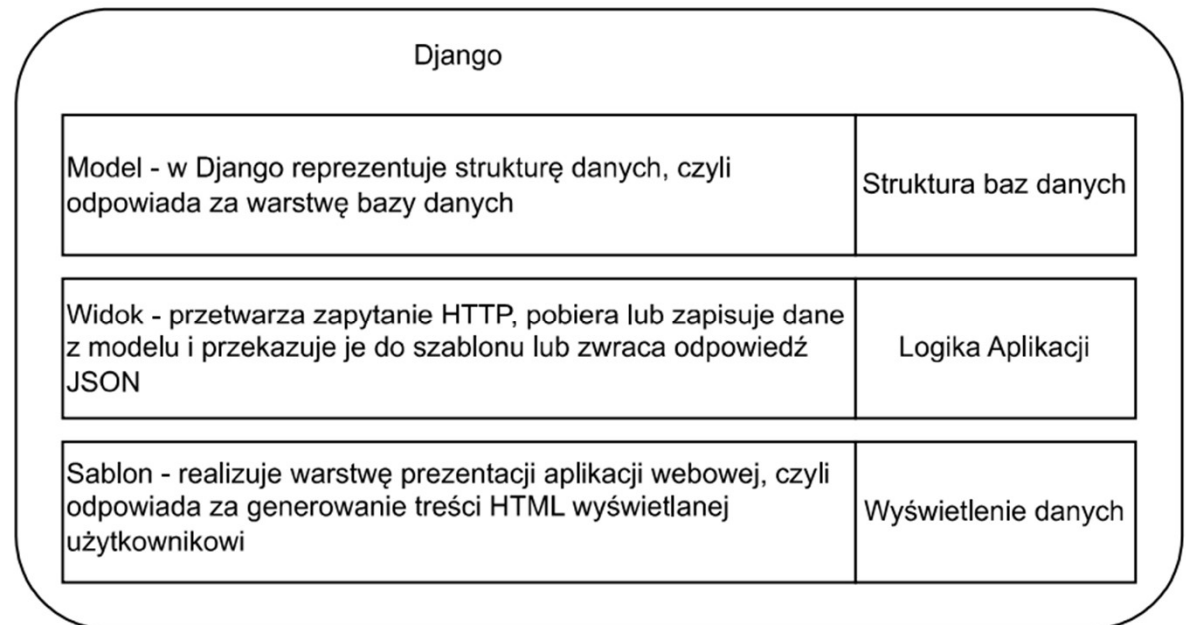
Architektura Systemu - Komunikacja



Architektura Systemu – ESP32

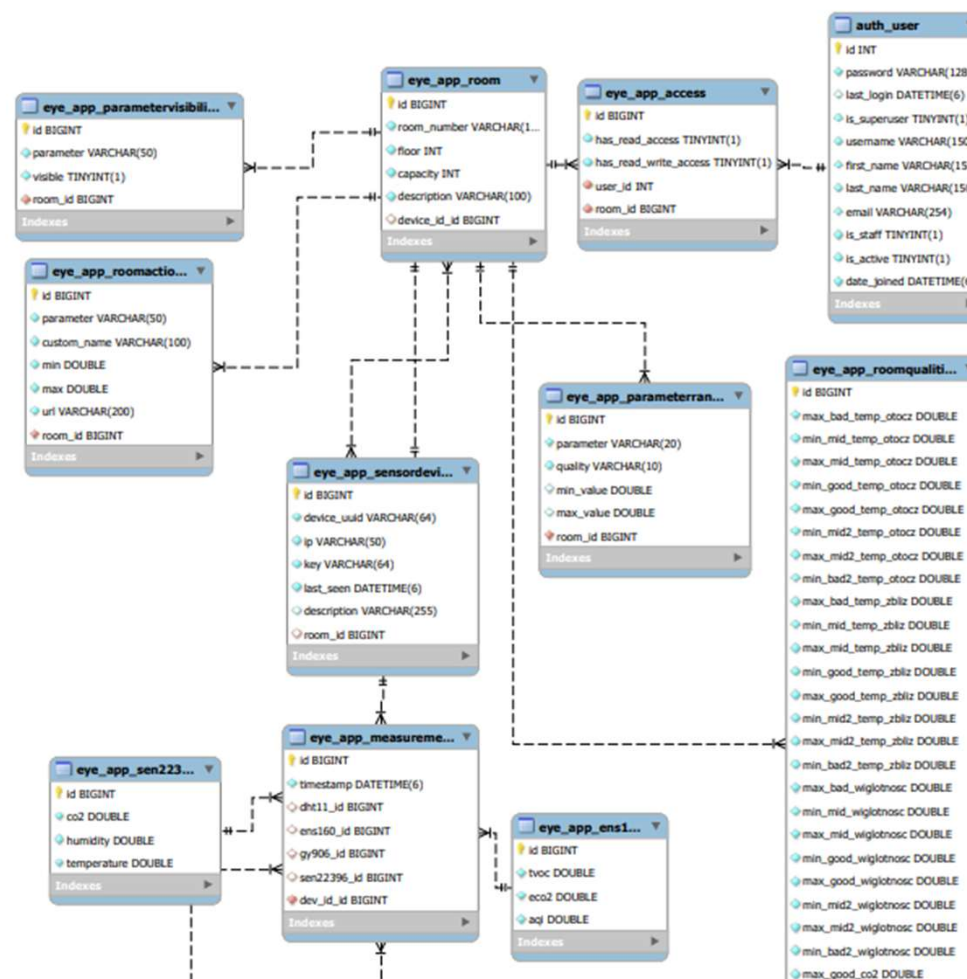


Architektura Systemu - Django



Architektura Systemu – Bazy Danych

Załącznik *EyeDiagramERD.pdf*



Komunikacja i bezpieczeństwo

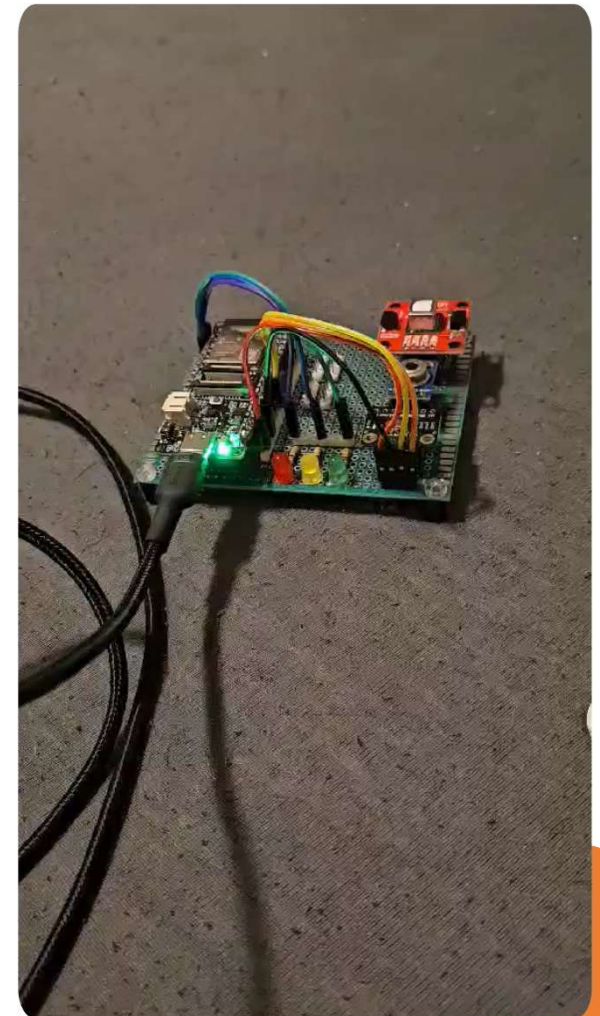
Komunikacja z czujnikami poprzez interfejs I2C

Komunikacja z serwerem poprzez sieć ESP32 po wifi łączy się z routerem Mikrotik, który kieruje ruch do OrangePi Zero3, na którym uruchomiono aplikację Django.



ESP32-S3 Użyte Biblioteki

- Preferences – zapisywanie danych w pamięci Flash
- Esp32ping – pingowanie urządzeń w sieci
- time – do podtrzymania czasu
- Aes – do szyfrowania
- string, sstream, iomanip, iostream,
- Arduino – zbiór podstawowych bibliotek
- Wire – do komunikacji I2c
- map – do tworzenia słowników
- WiFi – do ustanowienia połączenia z routerem
- HTTPClient – do wysyłania żądań do serwera Django
- SparkFun_SCD4x
- Adafruit_MLX90614
- DFRobot_ENS160
- thread – do pracy wielowątkowej





Interfejs szeregowy – Komunikacja z ESP32

Panel Pomocy – Pokazuje
możliwości odczytu oraz zmiany
parametrów

```
__ EYE_OS __  
  
? -> Get Some Help (This window)  
![0]{}# -> Print WiFi status  
![1]{}# -> Print Time Client status  
![2]{}# -> Print EYE APP status  
![3]{}# -> Print Device info  
![4]{}# -> Print Sensor status  
  
![a]{SSID}# -> Write WiFi SSID  
![b]{PASSWORD}# -> Write WiFi PASSWORD  
![c]{10.0.0.2}# -> Write EYE_APP IP  
![d]{8080}# -> Write EYE_APP PORT  
![e]{0 or 1}# -> Do you want to use encryption ?  
![$$]{UUID}# -> Write UUID to DEV  
![$$$]{KEY}# -> Write KEY to DEV  
![$$$$]{}# -> Delete KEY form DEV  
![r]{}# -> Reboot system
```


Interfejs szeregowy – Komunikacja z ESP32

Status połączenia WiFi

```

EYE_OS
WIFI_STATUS      -> CONNECTED
WIFI_SSID        -> Mikrotik-2G
WIFI_PASSWORD    -> *****
MAC_ADDRESS      -> 30196c188534
IP_ADDRESS       -> 10.0.13.214

```

Status połączenia z aplikacją

```

_ EYE_OS _
EYE_STATUS      -> CONNECTED
EYE_SERVER_IP   -> 10.0.13.219
EYE_SERVER_PORT -> 8000

```

Status synchronizacji czasu

```

_ EYE_OS _
TIME_STATUS      -> SYNC
TIME EPOCH MS    -> 1748781192265

```

Status czujników

__	EYE_OS
GY906	-> {"OBJ_TEMPERATURE":23.17,"AMB_TEMPERATURE":25.31}
ENS160	-> {"AQI":2.00,"TVOC":92.00,"ECO2":536.00}
SEN22396	-> {"CO2":712.00,"TEMPERATURE":23.49,"HUMIDITY":58.55}

Dane urządzenia

[illegible]

JSON przesyłany do serwera Django

Postać niezaszyfrowana

```
{
  "dev_uuid": "4a7c2f91b3e8d6cfc9a1be4fd3c2aa8e17f6a3b947ebd8c65b3f9d2484c7e12a",
  "dev_addr": "10.0.13.214",
  "dev_mac": "30196c188534",
  "time_epoch": 1748781824587,
  "command": "post",
  "data": {
    "sensors": [
      {
        "name": "ens160",
        "readings": {
          "aqi": 2,
          "tvoc": 108,
          "eco2": 563
        }
      },
      {
        "name": "sen22396",
        "readings": {
          "co2": 699,
          "temperature": 24.07,
          "humidity": 57.6
        }
      },
      {
        "name": "mlx90614",
        "readings": {
          "object_temperature": 24.77,
          "ambient_temperature": 25.99
        }
      }
    ]
  }
}
```

Postać zaszyfrowana

```
{
  "dev_uuid": "4a7c2f91b3e8d6cfc9a1be4fd3c2aa8e17f6a3b947ebd8c65b3f9d2484c7e12a",
  "time_epoch": 1748781620780,
  "dev_addr": "10.0.13.214",
  "command": "post",
  "encrypted": "1c6ee0085917ec947d822bd2a6dc160e63393815c257af3b338627913dfceaff4033e948f343a6d6b2cbf0315bec85332ac283fd1d2761569ab900628f2242411befb85ae020f7e2caa6901cf9de01020a1a8e13b9a42ae09962f3cb7df9510739c754c1442ff09ebb6606c802986796fb5bb83485aae3d5c2b41492693335d6873f6db1c9102f7ad98112b90dfd767daf482ee13c90fa979dd52702a97ccb3aa37f5c49899a27f9e56c312ab12c2f058ad9e1b895c85740a03913a12767c45360089691dbd75c6dd03959cb814a07d4b4fc6cf80f82589d30f714f44843c4960df69934feed96ce267773e7141531d633fbbc1c1a293a59d9159c0444c86340df0d50de00ec29a5d63a6f72c153a68aaf4d2e295cc8097e97ab52e59cf9f41a2a29f50e51d076ff1f924c9b022bf0a0"
}
```

Szyfrowanie danych przy pomocy AES-256

- Wykorzystujemy algorytm szyfrujący AES-256 w celu zabezpieczenia danych poprzez ich szyfrowanie i deszyfrowanie z pomocą 256 bitowego klucza. Klucz generowany jest na podstawie adresu UUID urządzenia, co zapewnia unikalność i powiązanie z konkretnym sprzętem.



shutterstock.com · 2364048597

Technologie backendowe użyte w projekcie:

Django (Python) – główny framework webowy.

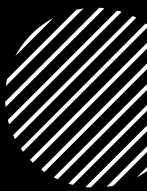
MySQL – relacyjna baza danych do przechowywania informacji o pokojach, urządzeniach, pomiarach i użytkownikach.

Django zarządza:

1. logiką aplikacji,
2. interakcją z bazą danych (ORM),
3. obsługą zapytań HTTP i routingiem,
4. uwierzytelnianiem użytkowników,
5. szablonami HTML (renderowanie widoków).



Funkcje realizowane przez Django:



Odbieranie danych pomiarowych z urządzeń (np. ESP32) przez API.



Dekodowanie i zapisywanie danych do bazy danych.



Obsługa widoków: główny panel, widoki pokoi, zarządzanie kontami i urządzeniami.



Generowanie stron HTML z dynamicznymi danymi.



Integracja z frontendem (HTML + JS + WebSockets w czasie rzeczywistym).

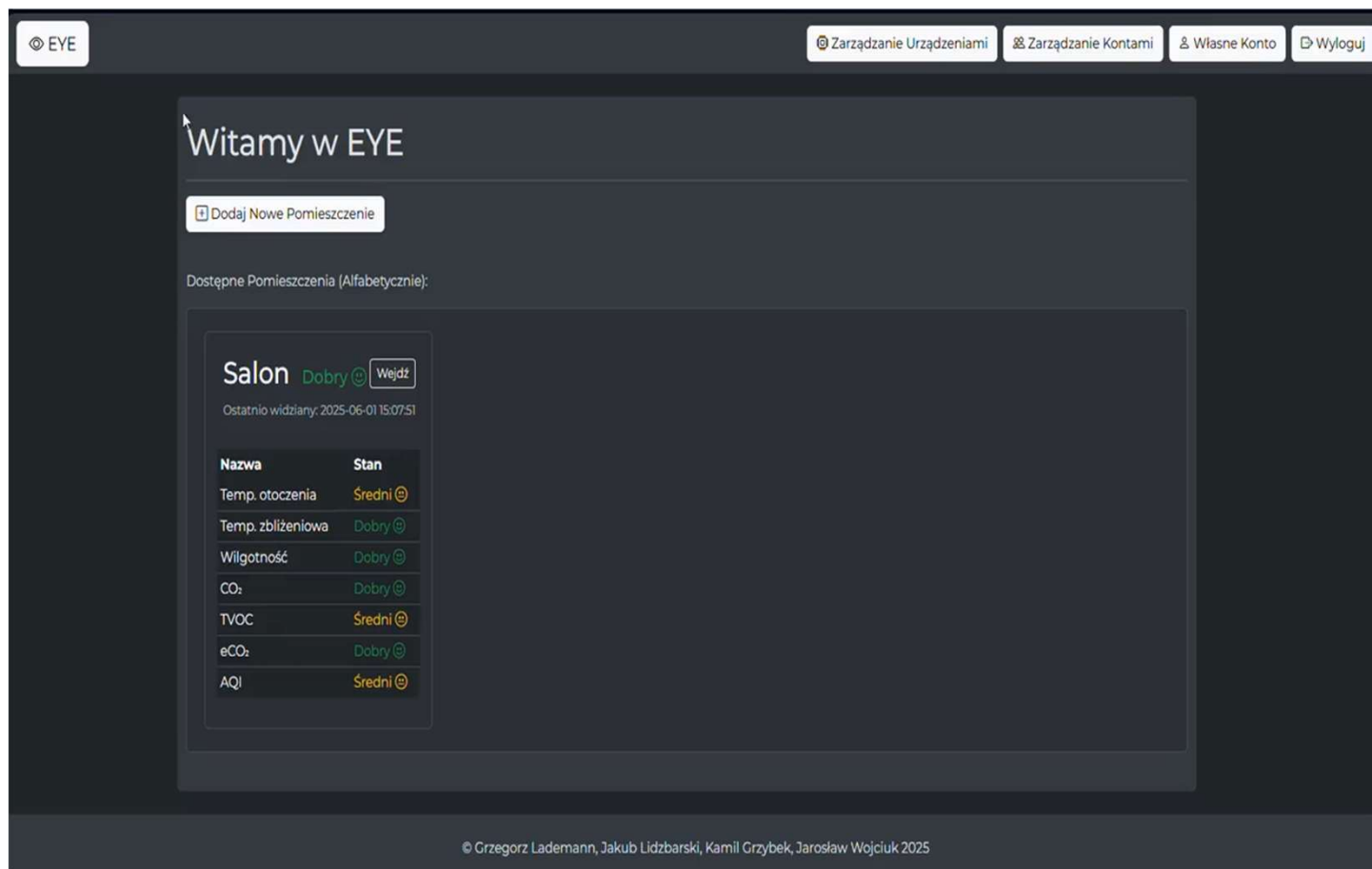


Zarządzanie sesją i autoryzacją użytkowników.

Kluczowe funkcje aplikacji:

-  **Logowanie i zarządzanie kontami** – role użytkowników, uprawnienia.
-  **Zarządzanie pokojami** – opisy, parametry, urządzenia.
-  **Pobieranie i prezentacja danych pomiarowych** – jakość powietrza, wykresy, statusy.
-  **Ustawienia zaawansowane** – pojęcia rozmyte, zakresy parametrów.
-  **Reakcje na przekroczenia progów** – akcje automatyczne (np. alerty).
-  **Deszyfrowanie odbieranych danych** – Odszyfrowywanie informacji z mikrokontrolera
-  **Dostęp do danych w czasie rzeczywistym**



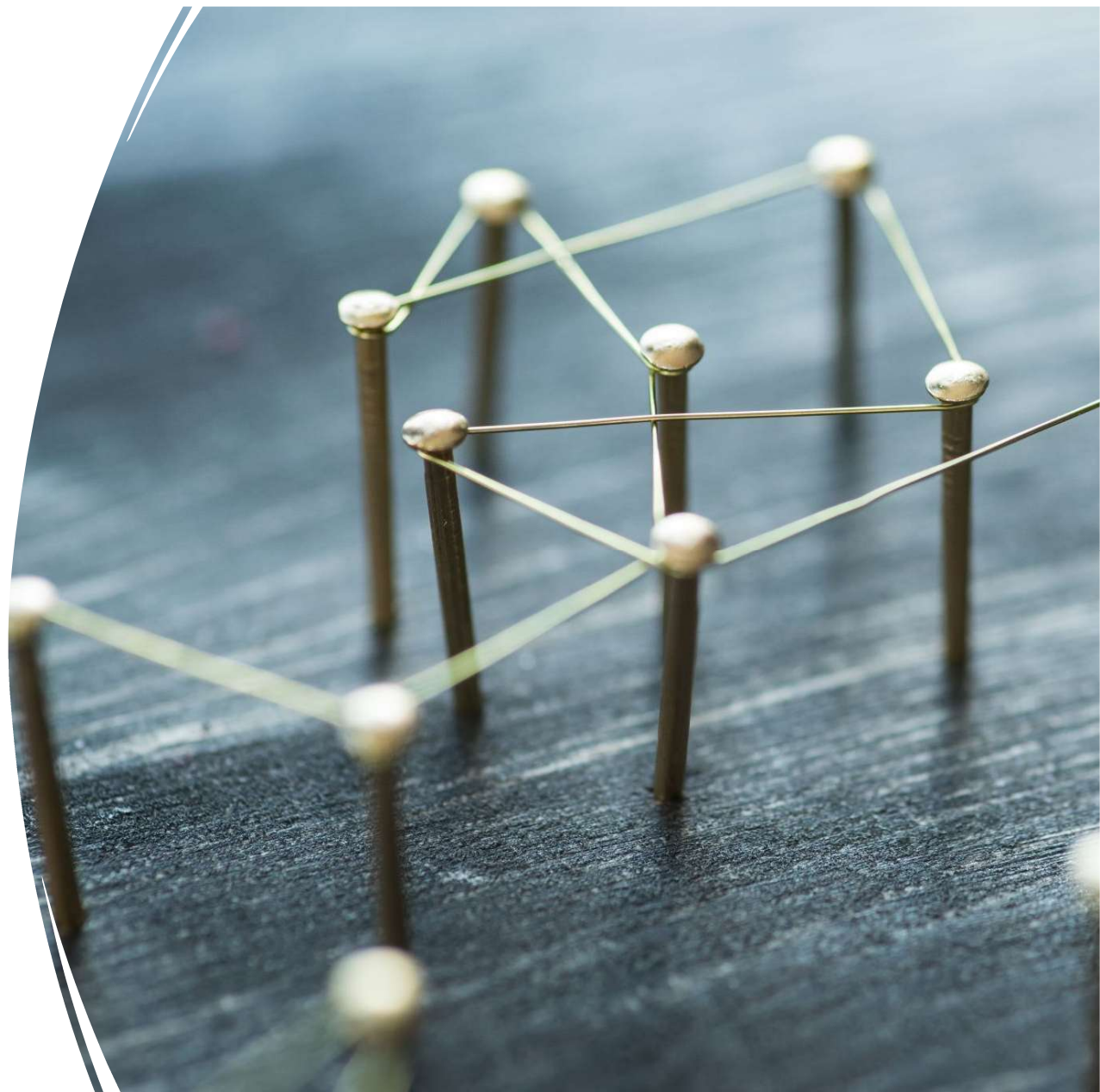


Działanie
aplikacji/strony

Załącznik
*EYE_strona_internetowa_k
ompresja.mp4*

Problemy i wyzwania

- Stabilność połączenia
- Translacja rozbudowanego systemu w czytelny interfejs
- (Zbędna) świadomość stref czasowych przez system
- Formatowanie wykresów
- Bezpieczne przekazywanie wrażliwych danych

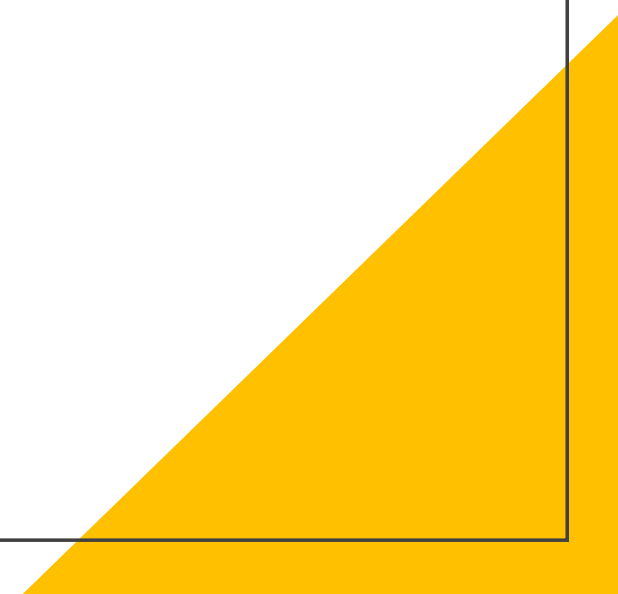


Możliwości rozwoju

- Definiowanie zdarzeń
- Modyfikacja przedziałów pojęć
- Poszerzenie systemu o większą ilość czujników
- Wydruk wykresów

Podsumowanie

- Wszystkie założenia projektowe zostały w pełni zrealizowane.
- Aplikacja działa zgodnie z oczekiwaniami: dane są odbierane, zapisywane i prezentowane.
- System obsługuje wiele urządzeń i użytkowników w czasie rzeczywistym.
- Udało się zrealizować wszystkie kluczowe funkcje: logowanie, konfiguracja pomieszczeń, akcje, wykresy i oceny jakości.
- Projekt został ukończony w terminie i przeszedł pomyślnie wszystkie testy funkcjonalne.
- System EYE jest gotowy do wdrożenia i dalszego rozwoju.



Załączniki

1. EyeDiagramERD.pdf
2. EYE_strona_internetowa_kompresja.mp4
3. esp32_praca.mp4
4. Dokumentacja Projektu Eye.docx
5. ESP32.pptx
6. EYE-StronaInternetowa.pptx
7. eye_github_repo.zip

